TransitPredict - Design Report

- Project Outline
- Software Requirements
    - Functional Requirements
    - Non-Functional Requirements
    - Deliberately Excluded
    - Future Enhancement
    - Acceptance Criteria
- System Architecture
    - Frontend Stack
    - Backend Stack
    - Data Ingestion
    - External Data Sources
    - Deployment
- Design Principles
    - Showing Real vs. Scheduled Data
    - Hub-Biased Display
    - Cache-First, Fail-Soft
    - Object Relation Model Principles Analysis
- Project Plan & Milestones
    - Week 15-16: Core Build
    - Week 17-18: Multi-Modal Integration
    - Week 19: Polish and Testing
    - Week 20: User Evaluation
    - Week 21: Final Submission
    - Week 22: Individual Submissions
- Task Allocation
    - Project Lead
    - Backend Developer A
    - Backend Developer B
    - Frontend Developer A: Core Interface
    - Frontend Developer B: Map and Visual Features
    - Data Analyst
- Risk Analysis
    - STOMP Connection Stability
    - Late Timetable Changes
    - Scope Creep
    - Team Member Availability
- Costings
    - Hardware costs
    - Software and services

        - Development tools
        - Hosting and infrastructure
        - API calls
    - Team
    - Summary
- Acceptance Tests
    - Core Functionality
    - Map and Location Features
    - Mobile and Accessibility
    - Performance and Reliability
- Activity Network
- Gantt Chart
- Appendix A: Class Diagrams for Services and Repositories
- Appendix B: Entity-Relation Diagram
- Appendix C: Database Table Relational Diagrams
- Appendix D: UI Mock-Up
    - Main menu
    - Train routing
    - Bus routing
    - Train search
    - Map search

Project Outline

The Lancashire coastal region between Preston and Lancaster, including Blackpool, the Fylde, and the Wyre coast, is served by six bus operators (Stagecoach Cumbria and North Lancashire, Stagecoach Merseyside and South Lancashire, Blackpool Transport, Archway Travel, Kirkby Lonsdale Coach Hire, and Transpora North West), multiple rail operators (Northern, Avanti West Coast, TransPennine Express), and the Blackpool tramway. Each of these provides timetable and tracking information independently through separate apps and websites, which creates a fragmented experience that discourages people from using public transport, especially for journeys that cross operator boundaries.

TransitPredict is our proposal to fix this. We are building a web application that pulls bus, rail, weather, and road data together into one reliable service for the region. All transport data is sourced from the university transport server at `transport.scc.lancs.ac.uk`, which provides cached, rate-limited copies of national datasets. Live rail information comes through STOMP subscriptions to the TRUST and Train Describer message feeds. Bus positions come from SIRI-VM GPS feeds polled every 15 seconds. Stations and stops are identified and located through the NaPTAN and NPTG databases, with cross-referencing handled by the CORPUS dataset, which maps between the various code systems used across the rail network (STANOX, TIPLOC, CRS, NLC).

The result is a system where a user can look up departures from any station or bus stop in the

area, see live vehicle positions on an interactive map, check the weather before heading out. Everything is labelled with how fresh the data is, so users always know whether they are looking at a live update or a scheduled timetable.

Software Requirements

Functional Requirements

- FR-01: Regional Coverage. The system shall cover rail stations and bus stops across the Preston to Lancaster coastal area, including Blackpool, the Fylde, and the Wyre coast.
- FR-02: Live Rail Departures. Display departures showing destination, scheduled and expected time, platform, operator, and status (on time, delayed, or cancelled) for any station in the coverage area. Data is sourced from the rail departures endpoint (by CRS code) with delay and cancellation detail enriched from the TRUST STOMP feed, specifically message types 0001 for activation, 0002 for cancellation, and 0003 for movement.
- FR-03: Data Freshness. Every piece of displayed data includes a last-updated timestamp. The interface warns users when data exceeds 3 minutes old and labels fallback information as “Scheduled” rather than presenting it as live.
- FR-04: Station and Stop Search. Users can search for rail stations by name or CRS code and bus stops by name or NaPTAN AtcoCode. Results are drawn from a pre-loaded NaPTAN dataset, filtered to the Lancashire coverage area (AtcoAreaCode 250 for bus stops, plus rail stations with CRS codes listed in the project brief).
- FR-05: Interactive Map. Display all covered stations and bus stops on an OpenStreetMap base layer with clickable markers that open departure information. Live bus positions are shown as moving markers updated every 15 seconds.
- FR-06: Delay Data Collection. Continuously log TRUST movement messages (type 0003) with their timetable variation and variation status fields throughout the development period (weeks 15 to 21) to build a dataset for delay trend visualisation and basic reliability scoring per station.
- FR-07: Regional Weather. Display current weather conditions for four points across the coverage area: Preston, Lancaster, Blackpool, and Morecambe, using the server weather endpoint.
- FR-08: Offline Fallback. When live feeds are unavailable, display cached scheduled timetable data clearly labelled as non-live, so that the application remains useful even during API outages.
- FR-09: Road Information. Display current National Highways VMS messages for the M6 corridor and major routes in the coverage area, showing travel time estimates and incident information.
- FR-10: Rail Station Facilities. Display facility information such as accessibility features, parking, toilets, and ticket office hours for each rail station, sourced from the station facilities endpoint.

Non-Functional Requirements

- NFR-01: Load Time. Pages load within 4 seconds on the university network under normal conditions.
- NFR-02: Mobile-First. The application is fully functional from 360px screen width upward, with core features accessible on any modern mobile browser.
- NFR-03: Polling Limits. Bus live position polling is capped at one request per operator per 15 seconds. Rail departure requests are cached for 30 seconds per station. Weather requests are cached for 5 minutes per location. All limits are designed to stay within the rate-limited proxy on the university server.
- NFR-04: Accessibility. The application meets WCAG 2.1 AA standards for colour contrast, keyboard navigation, and screen reader compatibility.
- NFR-05: Concurrent Users. The system supports 50 concurrent users under normal operating conditions, verified through load testing during development.
- NFR-06: Deployment. Primary deployment uses Podman containers on university lab infrastructure, with the option to deploy the frontend to Vercel for external access during user evaluation.
- NFR-07: Documentation. The codebase includes a setup README, inline comments for complex logic, and all AI-generated code is clearly labelled per module policy.

Deliberately Excluded

- EX-01: Ticket Purchase. This requires commercial partnerships with operators which are beyond the scope of a student project.
- EX-02: Full Journey Planner. While the system displays bus and rail departures from the same interface and the map shows both transport modes together, calculating optimal transfer times between bus stops and rail stations with walking distance estimates is beyond the current scope. The interface does surface co-located bus stops and rail stations to help users plan interchanges manually.
- EX-03: User Accounts. Adding login functionality would introduce data protection obligations and authentication complexity that would take time away from the core transport features.
- EX-04: Push Notifications. This would require native mobile app development or a service worker implementation that we have deprioritised in favour of reliable core functionality.
- EX-05: Advanced ML Prediction. The data collection period (roughly 6 weeks of live delay logging) is too short to train meaningful predictive models. We focus on descriptive statistics and trend visualisation instead.
- EX-06: Tram-Specific Integration. Blackpool Transport tram services already appear in the BLAC operator feed and will be displayed alongside bus departures. A dedicated tram tracking view with route visualisation is excluded but the underlying data is present for future extension.

Future Enhancement

The following features are straightforward extensions of the current architecture and could be delivered at a later date.

- **Temporary Speed Restriction display**. TSR data arrives weekly via the STOMP feed and is already received by our client. Rendering it on the map would require only a new marker layer.
- **Station-to-station delay history**. The delay logger already captures per-station variation data. Aggregating this into average delay between two stations by time of day would be a single database query and a chart component.
- **Departure board for any UK station**. The rail departures endpoint works for any CRS code nationally. Expanding beyond Lancashire would be a configuration change rather than a code change.
- **VSTP live timetable amendments**. We already subscribe to the STOMP feed. Adding a handler for VSTP messages to update the local schedule cache in real time would be an incremental addition to the existing message processing pipeline.
- **Rail schedule visualisation**. The daily schedule data contains full journey plans for every service. Rendering a timeline view for a selected service would reuse existing timetable data with a new frontend component.

Acceptance Criteria

- AC-01: The system covers at least 20 rail stations in the Preston to Lancaster corridor, including all CRS codes listed in the project brief.
- AC-02: Searching “Lanc” returns Lancaster in the top 3 results within 1 second.
- AC-03: The departure board for Lancaster (CRS: LAN) matches National Rail live departure data within 60 seconds of a TRUST movement message being received.
- AC-04: Data timestamps show updates within 2 minutes during normal operation.
- AC-05: Clicking a map station marker loads the departure board within 3 seconds.
- AC-06: The mobile layout at 375px width works without horizontal scrolling and all interactive elements are reachable.
- AC-07: When APIs fail, the interface shows a “Scheduled Data” fallback with a visible warning banner.
- AC-08: A bus departure board for a Preston city centre stop shows services from at least two different operators in a single unified view.

System Architecture

Frontend Stack

Next.js with TypeScript for type safety and server-side rendering. Tailwind CSS for responsive styling. Leaflet.js with OpenStreetMap tiles for interactive mapping. Local storage for user preferences like favourite stations and recent searches, avoiding the need for backend user

management.

Backend Stack

Node.js with Express for REST API endpoints. PostgreSQL for persistent storage of station metadata from NaPTAN and CORPUS, parsed timetable data, and collected historical delay records. Redis for short-lived caching of live departure data and bus positions. A dedicated STOMP client process maintains persistent connections to the transport server.

Data Ingestion

On startup or on a weekly schedule, the system parses and loads the NaPTAN and NPTG XML files into PostgreSQL, indexing by AtcoCode and locality reference. It parses the CORPUS JSON to build a lookup table mapping between STANOX, TIPLOC, CRS, and NLC codes. It loads the SMART dataset to map Train Describer berth movements to station arrivals and departures. It decompresses and parses the daily rail schedule, extracting services that call at stations within the coverage area.

Continuously, the STOMP client receives TRUST messages. These are filtered by region, keeping messages where the STANOX falls in the North West ranges or where the TD area code is relevant. Events are written to Redis for live state and to PostgreSQL for historical logging. In parallel, background workers poll the bus live endpoint for each of the six operators every 15 seconds and update the bus position cache. Weather is polled for four regional coordinates every 5 minutes. Road VMS data is polled every 2 minutes.

When a user makes a request, the frontend hits the Express API, which assembles a response from Redis for live data and PostgreSQL for timetable and metadata. If live data is unavailable or stale, the response falls back to scheduled data with an explicit staleness warning attached to the payload.

External Data Sources

All data flows through transport.scc.lancs.ac.uk. The specific endpoints and feeds used are: rail departures by CRS code, station facilities by CRS code, the BPLAN file for network topology, the CORPUS file for location code lookups, the SMART file for berth-to-station mapping, the daily schedule file, and the published working timetable. Live data comes from STOMP subscriptions to the TRUST train movement feed, the TD signalling feed, the VSTP timetable update feed, and the TSR feed. Bus data comes from the timetable endpoint and the live GPS endpoint, both accessed by operator NOC code. Weather comes from the latitude/longitude weather endpoint. Road information comes from the VMS endpoint. Station and stop metadata comes from the NaPTAN and NPTG XML files.

Deployment

Backend services (the Express server, STOMP client, PostgreSQL, and Redis) run as Podman containers on university lab machines, communicating over a private container network. Persistent volumes are mapped to H-Drive storage so that data survives container restarts. The frontend is deployed to Vercel for accessibility during user evaluation. Only TCP ports in the 5000-5100 range are used for inter-machine communication on the lab network, as specified in the project brief.

Design Principles

Showing Real vs. Scheduled Data

Every piece of data shown to the user carries a timestamp and a source label. If a bus position is 45 seconds old, the interface says so. If the rail departure board is built from the scheduled timetable because the TRUST feed has not yet reported on a particular service, it is labelled “Scheduled” rather than “On time.” We looked at how existing apps sometimes display confident arrival estimates that turn out to be stale, and decided that honesty is more important than reassurance. Users can make better decisions when they know how current the information actually is.

Hub-Biased Display

The brief raises a specific concern about users being stranded at remote stops after missing a connection. Our interface addresses this by showing departures from Preston and Lancaster as fallback options whenever someone views a smaller station. If a user is looking at Kirkham and Wesham departures, a sidebar shows the next departures from Preston too, because Preston has significantly more frequent services and is one stop away. A slightly longer journey through a well-served hub is preferable to a tight connection at a station with hourly service.

Cache-First, Fail-Soft

Redis sits between every external data request and the user. Bus positions are cached with a 15-second TTL, rail departures with 30 seconds, and weather with 5 minutes. If an upstream source fails, we serve the cached version with a visible age indicator rather than showing an error page. The STOMP feed is push-based so there is no polling to fail, but if the connection drops, the client reconnects with exponential backoff, and the frontend falls back to the HTTP departure endpoint until the stream recovers.

Object Relation Model Principles Analysis

- **Single Responsibility Principle (SRP)**: We designed each class in the system to only have one single, well-defined responsibility. The ApiIngestionService, for example, is

responsible only for ingesting and normalising external transport data, while the ApiHealthService focuses solely on monitoring API availability and latency. RouteVerificationService is set up to cross-check route data across multiple sources, and ReliabilityService is responsible only for computing reliability scores. In the database model, the entities such as Route, Station, and ServiceStatus each represent one distinct domain concept. Reducing any overlap to the minimum.

- **Open–Closed Principle (OCP)**: The entities and services are structured so that new functionality can be added later without modifying existing code. For example, new transport data providers can be added by implementing the TransportApiClient interface without changing the ingestion or verification services. Extra reliability factors can be added by extending the logic in ReliabilityService rather than rewriting it. This allows our system to scale while maintaining stability. Hence, the open-closed principle.
- **Liskov Substitution Principle (LSP)**: All implementations of the TransportApiClient interface can be substituted without affecting system behaviour. Services such as ApiIngestionService depend on the interface rather than on specific external API implementations. This allows different transport providers to be swapped without impacting the ingestion process.
- **Interface Segregation Principle (ISP)**: Interfaces and repositories only access the functionality required by the services that use them. For example, ReliabilityService depends only on ReliabilityRepository and VerificationRepository and does not depend on, say, the ingestion-related repositories. Then you can also take the TransportApiClient interface, which contains only methods relevant to retrieving transport data. This setup avoids unnecessary dependencies and keeps the services loosely coupled.
- **Dependency Inversion Principle (DIP)**: All the high-level services depend on the abstractions rather than the actual concrete implementations. Core logic components only interact with the repositories or the API clients through interfaces instead of directly accessing databases or external APIs. For instance, RouteVerificationService uses the VerificationRepository rather than directly interacting with SQL code. This separation improves testability, maintainability, and flexibility of the system architecture.

Project Plan & Milestones

Week 15-16: Core Build

STOMP client connected and filtering TRUST messages for coverage-area stations. Rail departure boards working for all stations in the brief using the departure endpoint with TRUST enrichment. NaPTAN and CORPUS data loaded into PostgreSQL with search indexing. Interactive map showing station markers with click-through to departure boards. Mobile-responsive departure board UI. Delay data logging running in the background from the start. Deliverable for week 17 demo: working live rail departure boards and interactive map demonstrated in the lab.

Week 17-18: Multi-Modal Integration

Bus timetable parsing complete for all six operators. Bus live GPS polling running and bus positions displayed on the map. Bus departure boards showing estimated arrivals based on GPS data combined with timetable. Weather overlay for four regional points. Road VMS information display for M6 corridor. Station facilities display. Historical delay data collection continuing. Deliverable for week 19 demo: integrated bus and rail system with weather and road data, demonstrated in the lab.

Week 19: Polish and Testing

Performance testing and optimisation. Accessibility audit against WCAG 2.1 AA. Security review of all dependencies and API credential handling. Edge case handling for missing data fields, operator inconsistencies, and network interruptions. Load testing with 50 simulated concurrent users.

Week 20: User Evaluation

Ethics-approved user study with 8 to 12 participants. Task-based evaluation covering finding the next bus, checking rail delays, and interpreting weather and road information. System Usability Scale (SUS) questionnaire and semi-structured follow-up interviews. Results analysis and write-up.

Week 21: Final Submission

Code submission via group repository with inline documentation. Presentation video including system demo and technical walkthrough. All AI-generated code clearly labelled and attributed per module policy.

Week 22: Individual Submissions

Security self-assessment against the NCSC Software Security Code of Practice. Peer assessment. Self-reflective report.

Task Allocation

We have divided work across the team based on six broad roles. In practice, everyone contributes across areas, but each person has primary ownership of their tasks.

Project Lead

Setting up the shared repository with branching strategy and CI/CD pipeline. Establishing team communication protocols and weekly progress tracking. Managing API credentials securely. Coordinating integration between backend and frontend work. Overseeing the final deployment and go-live process. Preparing progress reports and presentation materials. Leading the security review in week 19.

Backend Developer A

Design and implementation of the system's data models, ensuring alignment with the relational database schema and maintaining proper normalisation. An API ingestion layer was developed along with scheduled cron jobs to periodically retrieve and persist transport data from external sources. A route reliability calculator was implemented to compute reliability scores based on historical performance and verification results. Additionally, a route cross-checking component was developed to compare route data across multiple providers and generate confidence scores.

Backend Developer B

Development of route search logic to enable efficient transport data queries. Administrative API endpoints were implemented to support system management tasks. Authentication and role-based access controls were introduced to secure administrative functionality. In addition, caching mechanisms were configured to enhance performance and reduce database overhead.

Frontend Developer A: Core Interface

Setting up the Next.js project with TypeScript and Tailwind CSS. Building the component library for departure boards, search, status indicators, and data freshness labels. Implementing the home page with transport mode selection. Building the station and bus stop search with autocomplete, drawing results from the backend search endpoint. Building the departure board display that handles both rail and bus services in a unified format. Implementing automatic data refresh on a 30 to 60 second cycle with visible refresh indicators. Handling the offline fallback display with appropriate warning banners.

Frontend Developer B: Map and Visual Features

Integrating Leaflet.js with OpenStreetMap tiles. Rendering station and bus stop markers from NaPTAN coordinate data. Implementing live bus position markers that update every 15 seconds with smooth interpolation between positions. Building the weather overlay with toggle controls. Implementing marker clustering so the map performs well when zoomed out. Building the hub-biased sidebar that shows Preston or Lancaster departures alongside smaller station views. Optimising map performance on mobile devices.

Data Analyst

Designing the delay data collection pipeline that logs TRUST 0003 messages to PostgreSQL with train ID, location STANOX, planned timestamp, actual timestamp, timetable variation, variation status, and operator ID. Writing daily aggregation queries for average delay per station, percentage on time per operator, and delay distribution by hour of day. Building a simple reliability score per station calculated as on-time departures divided by total departures over a rolling 7-day window. Building the analytics display components for delay trends and reliability scores. Leading the user evaluation in week 20, including ethics application, participant recruitment, consent forms, task design, SUS administration, and results analysis.

Risk Analysis

STOMP Connection Stability

The persistent STOMP connection may drop due to network interruptions, server maintenance, or VPN disconnection when working off campus. If the connection drops silently, users would see stale data presented as if it were current, which directly contradicts our transparency principle. We mitigate this through heartbeat monitoring. The messages arrive periodically on the feed, so if we stop receiving them, we know the connection has failed. The client reconnects automatically with exponential backoff. During disconnection, the frontend falls back to the HTTP rail departures endpoint and labels all data as “Scheduled” until the STOMP stream recovers. A visible banner tells users that the live feed is recovering.

Late Timetable Changes

Very Short-Term Planning updates can amend the rail schedule within 48 hours of a train running, adding services, cancelling services, or changing stopping patterns. If we only load the daily schedule at startup, we could miss these changes and show incorrect departure information. We mitigate this by subscribing to the VSTP_ALL topic on the STOMP feed. Processing VSTP messages to update the local schedule cache is planned for the week 17 to 18 development sprint, after the core TRUST integration is stable. Until then, the HTTP departure endpoint provides VSTP-aware data as a fallback, so users still see correct information even before our own VSTP processing is live.

Scope Creep

The project brief describes many data sources and potential features, and it would be easy to keep adding functionality beyond what we have committed to delivering. The risk is that we spread ourselves too thin and deliver nothing properly rather than a solid core product. We handle this by having a clear list of what is excluded and what is noted for future work. The team lead reviews any proposed additions against the timeline before they are started. New

ideas go into the future enhancements list rather than the current sprint unless we are genuinely ahead of schedule.

Team Member Availability

With six people and a 12-week development window, losing one person to illness or other commitments could create significant delays, especially if they own a critical component. We mitigate this through documentation and pair working. The STOMP client and the bus parser are the two most specialised components, so both backend developers attend each other's code reviews and can pick up the other's work at short notice. All work is committed to the shared repository daily, and no one keeps work-in-progress only on their local machine.

Costings

Hardware costs

Item	Cost (£)	Notes
Work laptops	3,724.98	M1 MacBooks for every team member

Software and services

Development tools

Item	Cost (£)	Notes
VSCode	0.00	Free tier (commercial use permitted)
Git	0.00	Free for commercial use
GitHub	0.00	Free tier (commercial use permitted)
Figma	0.00	Starter plan (commercial use permitted)
Postman	0.00	Free for commercial use
pgAdmin	0.00	Free for commercial use

Hosting and infrastructure

Item	Ongoing cost (£/ unit)	Total cost[1] (£)	Notes
Railway incl. Railway PostgreSQL	14.72 /month	529.92	Backend hosting

Item	Ongoing cost (£/ unit)	Total cost[1] (£)	Notes
Upstash Redis	0.15 /100k commands	16.12	Caching (assumes 10k commands per day)
GitHub Actions	N/A, expected use falls within limits	0.00	CI/CD
GitHub Releases	N/A, unlimited use	0.00	Electron distribution
.co.uk domain name	8.00 /year	24.00	

API calls

Item	Ongoing cost (£/unit)	Total cost[1] (£)	Notes
Network Rail APIs	N/A, unlimited use	0.00	
OpenWeatherMap	N/A, expected usage falls within limits	0.00	
National Highways APIs	N/A, unlimited use	0.00	

Team

Role	Count	Hourly rate (£)	Hours/week each	12 week total cost (£)
Project Manager	1	55.00	40	26,400.00
Backend Developer	2	50.00	40	48,000.00
Frontend Developer	2	50.00	40	48,000.00
Data Analyst	1	50.00	40	24,000.00

Summary

Category	Cost (£)
Hardware	3,724.98
Development	0.00
Hosting	570.04
APIs	0.00
Team	146,400.00
Subtotal	*150,695.02*
Overhead (20%)	30,139.00

Category	Cost (£)
Profit (25%)	37,673.76
Total	**218,507.78**

Acceptance Tests

Core Functionality

- AT-01: Station and Bus Stop Search. A user can search for any rail station in the coverage area by typing part of its name or its CRS code. Bus stops in Preston, Lancaster, and Blackpool are searchable by location name. Results appear as the user types, with transport mode icons distinguishing rail stations from bus stops.
- AT-02: Live Rail Departure Display. The rail departure board for a selected station shows destination, scheduled time, expected time, platform number, operator name, and status. The display updates automatically every 30 seconds. A departure that is running 5 minutes late shows an amber status with the delay duration. A cancelled service shows red status with the word “Cancelled.”
- AT-03: Live Bus Departure Display. The bus departure board for a selected stop shows route number, destination, operator name, and expected arrival time. Where live GPS data is available, the expected time is calculated from the vehicle position. Where GPS data is not available, the timetabled time is shown and labelled “Scheduled.”
- AT-04: Data Freshness. Every departure board shows a “Last updated” timestamp. When data is older than 3 minutes, a yellow warning appears. When live feeds are unavailable, the board switches to scheduled data with a clear “Showing scheduled timetable data” banner.
- AT-05: Multi-Operator Bus View. A bus stop in Preston city centre that is served by both Stagecoach, and another operator displays services from all operators in a single combined list, sorted by expected arrival time.

Map and Location Features

- AT-06: Interactive Map. The map renders the Preston to Lancaster corridor with markers for all covered rail stations and bus stops. Station markers and bus stop markers are visually distinct. Clicking a marker opens the departure board for that location. The map performs smoothly during pan and zooms on a mid-range mobile device.
- AT-07: Live Bus Positions. Bus positions from the SIRI-VM feeds for all six operators appear on the map as markers showing the route number. Positions update every 15 seconds. Markers move smoothly between updates rather than jumping. Vehicles outside the coverage area are filtered out.
- AT-08: Weather and Road Information. Current weather for Preston, Lancaster, Blackpool, and Morecambe display temperature, conditions description, and an appropriate weather

icon. Road VMS messages for the M6 corridor display journey time estimates. Both the weather layer and the road information can be toggled on and off on the map.

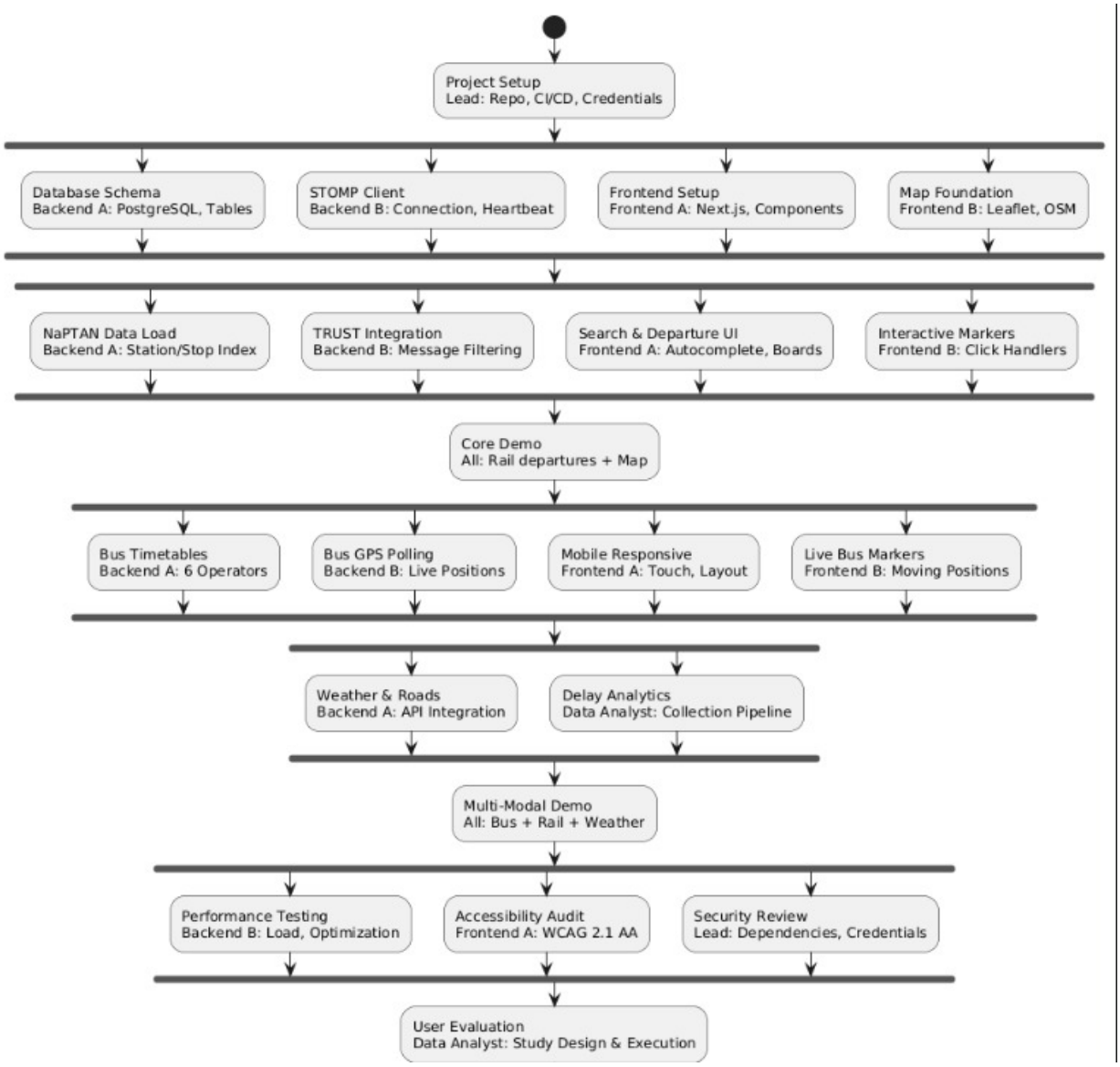
Mobile and Accessibility

- AT-09: Mobile Responsiveness. The application is fully functional on a 360px wide screen. Touch targets for all interactive elements meet the minimum 44px size. The navigation works without horizontal scrolling. The departure board is readable without zooming.
- AT-10: Accessibility. All interactive elements are reachable by keyboard. Screen readers correctly announce station names, departure times, and service status. Colour contrast meets WCAG 2.1 AA ratios. Status information (on time, delayed, cancelled) is conveyed through text labels as well as colour, so it is not dependent on colour perception alone.

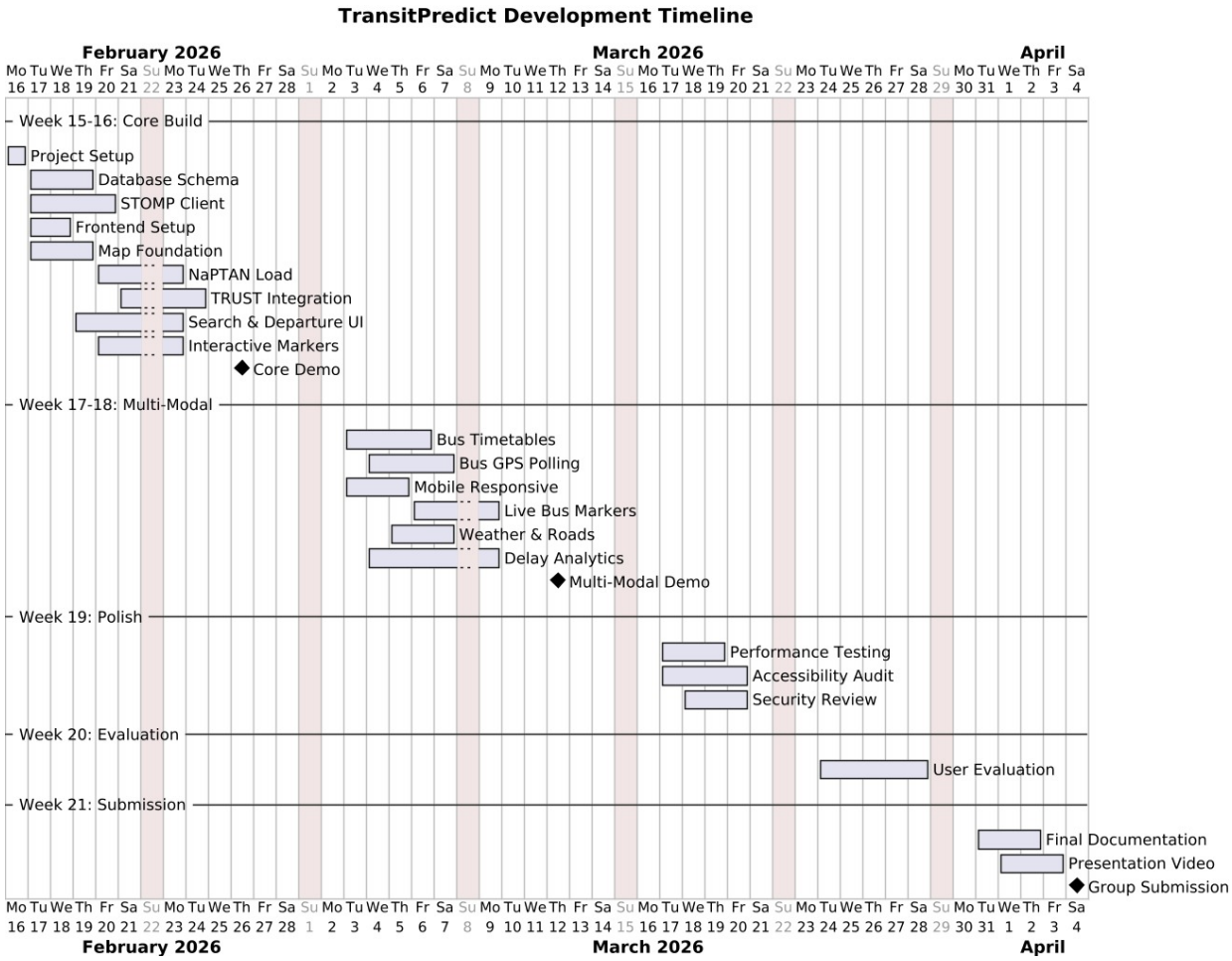
Performance and Reliability

- AT-11: Load Performance. The departure board for any station loads within 4 seconds on the university network. The map renders with all markers within 5 seconds on initial load. Subsequent data refreshes complete within 2 seconds.
- AT-12: Error Handling. If the STOMP connection drops, the system falls back to HTTP departure data within 30 seconds and displays a recovery banner. If the bus live feed for an operator returns an error, the departure board shows timetabled times for that operator’s services with a “Scheduled” label. Invalid search queries return a helpful message suggesting corrections.

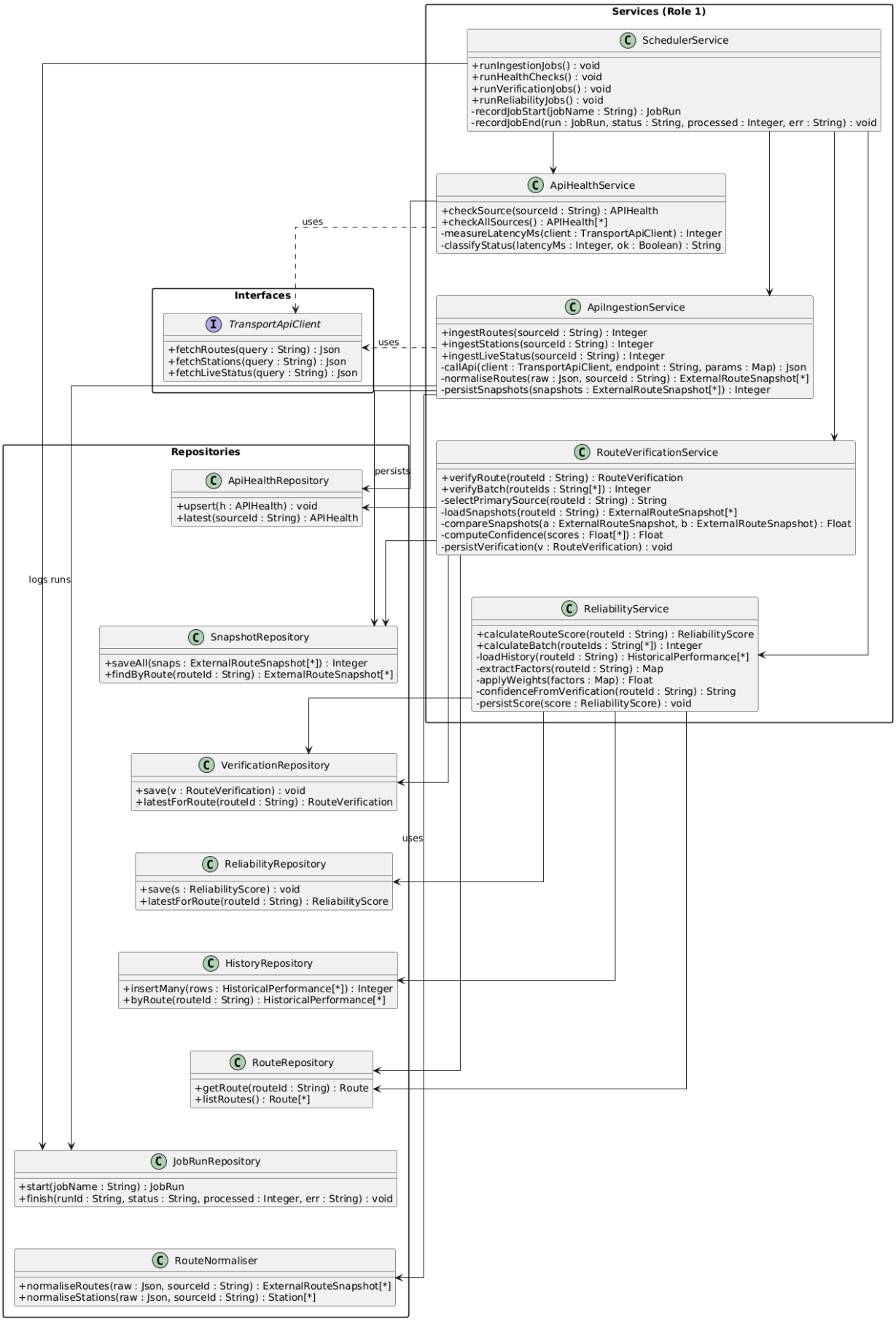
Activity Network


Project Setup
Lead: Repo, CI/CD, Credentials
Database Schema
Backend A: PostgreSQL, Tables
STOMP Client
Backend B: Connection, Heartbeat
Frontend Setup
Frontend A: Next.js, Components
Map Foundation
Frontend B: Leaflet, OSM
NaPTAN Data Load
Backend A: Station/Stop Index
TRUST Integration
Backend B: Message Filtering
Search & Departure UI
Frontend A: Autocomplete, Boards
Interactive Markers
Frontend B: Click Handlers
Core Demo
All: Rail departures + Map
Bus Timetables
Backend A: 6 Operators
Bus GPS Polling
Backend B: Live Positions
Mobile Responsive
Frontend A: Touch, Layout
Live Bus Markers
Frontend B: Moving Positions
Weather & Roads
Backend A: API Integration
Delay Analytics
Data Analyst: Collection Pipeline
Multi-Modal Demo
All: Bus + Rail + Weather
Performance Testing
Backend B: Load, Optimization
Accessibility Audit
Frontend A: WCAG 2.1 AA
Security Review
Lead: Dependencies, Credentials
User Evaluation
Data Analyst: Study Design & Execution

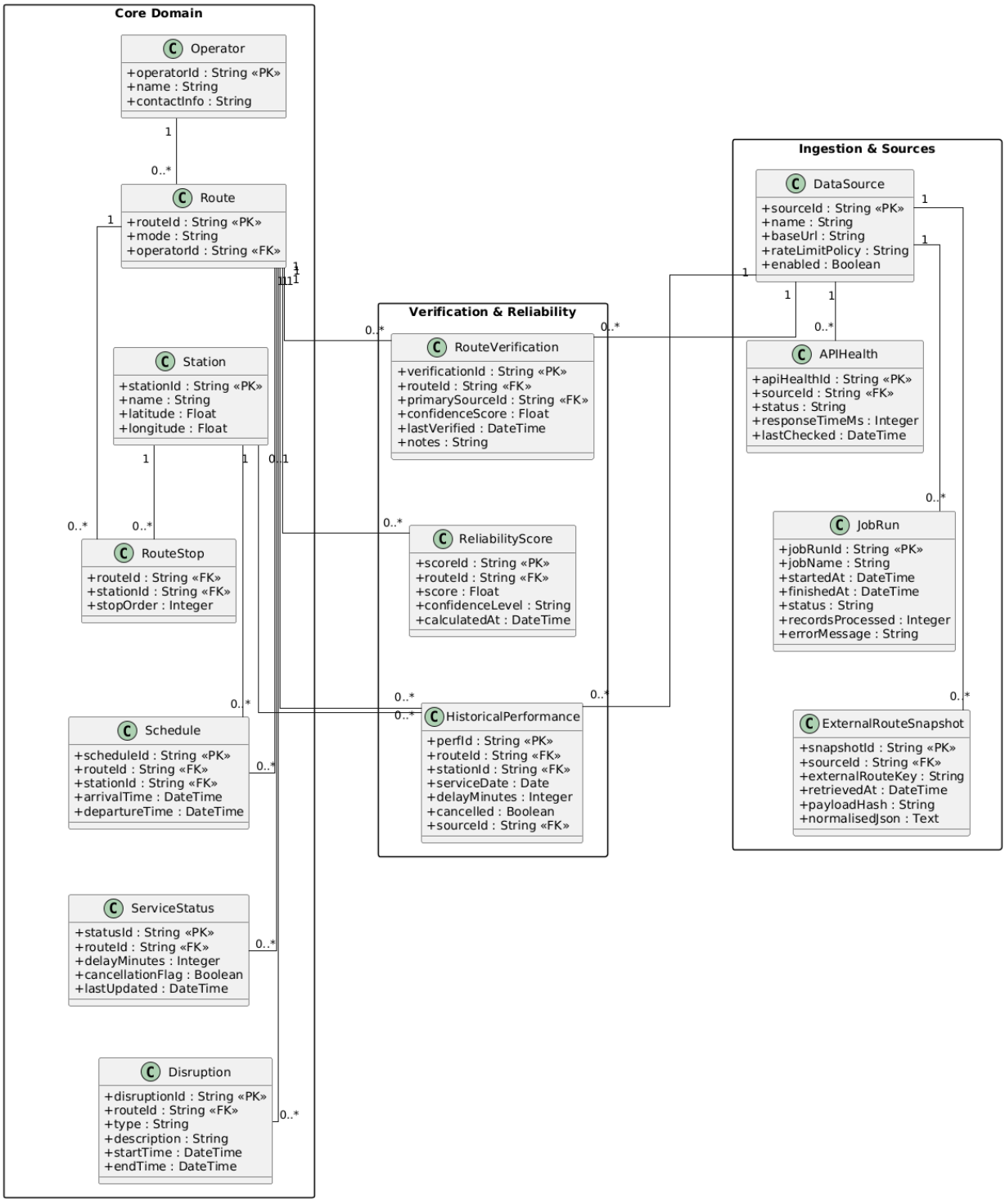
Gantt Chart



Appendix A: Class Diagrams for Services and Repositories

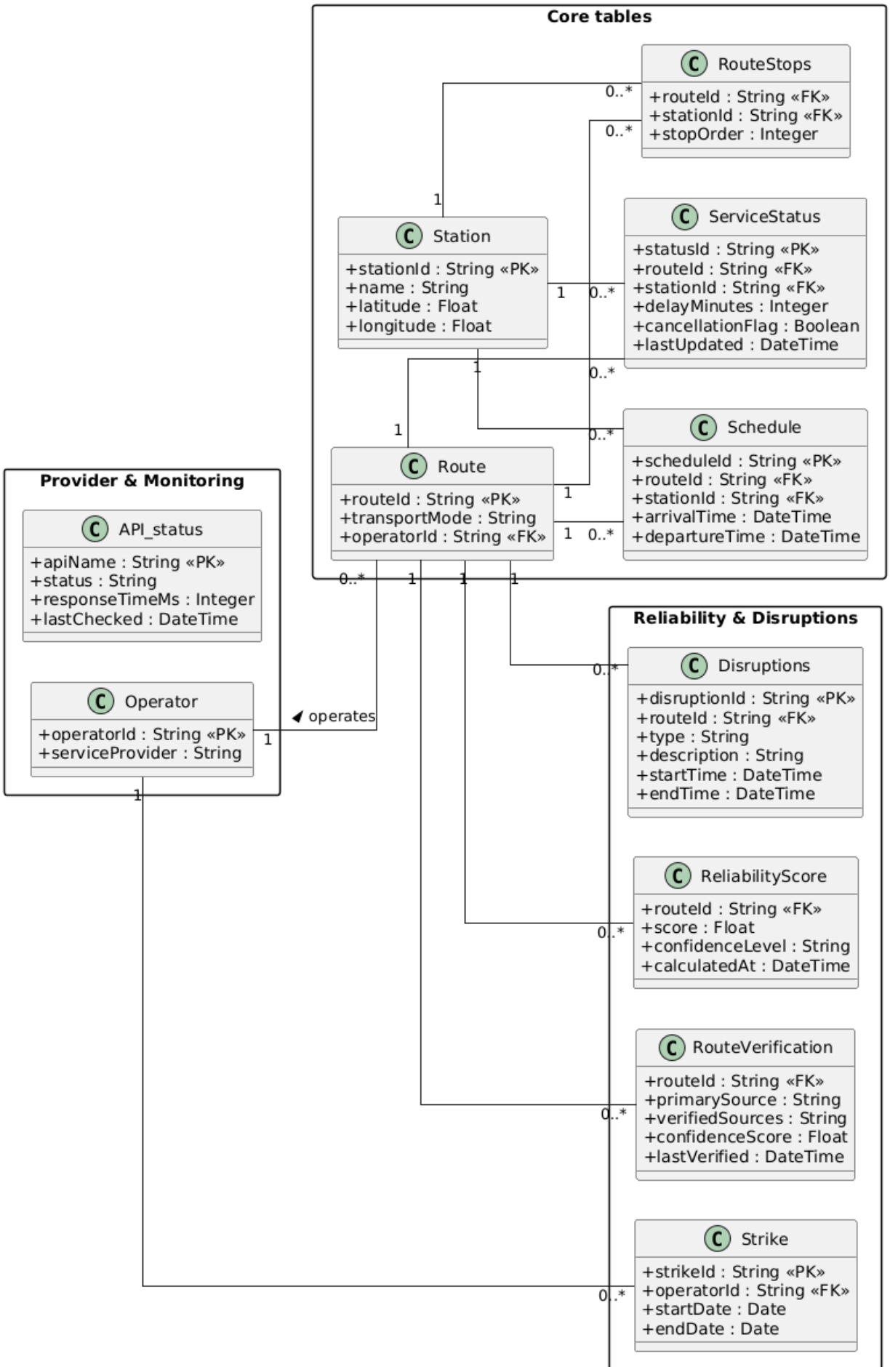

Services (Role 1)
SchedulerService
+runIngestionJobs() : void
+runHealthChecks() : void
+runVerificationJobs() : void
+runReliabilityJobs() : void
-recordJobStart(jobName : String) : JobRun
-recordJobEnd(run : JobRun, status : String, processed : Integer, err : String) : void
ApiHealthService
+checkSource(sourceId : String) : APIHealth
+checkAllSources() : APIHealth[*]
-measureLatencyMs(client : TransportApiClient) : Integer
-classifyStatus(latencyMs : Integer, ok : Boolean) : String
uses
Interfaces
TransportApiClient
+fetchRoutes(query : String) : Json
+fetchStations(query : String) : Json
+fetchLiveStatus(query : String) : Json
uses
ApiIngestionService
+ingestRoutes(sourceId : String) : Integer
+ingestStations(sourceId : String) : Integer
+ingestLiveStatus(sourceId : String) : Integer
-callApi(client : TransportApiClient, endpoint : String, params : Map) : Json
-normaliseRoutes(raw : Json, sourceId : String) : ExternalRouteSnapshot[*]
-persistSnapshots(snapshots : ExternalRouteSnapshot[*]) : Integer
Repositories
ApiHealthRepository
+upsert(h : APIHealth) : void
+latest(sourceId : String) : APIHealth
persists
RouteVerificationService
+verifyRoute(routeId : String) : RouteVerification
+verifyBatch(routeIds : String[*]) : Integer
-selectPrimarySource(routeId : String) : String
-loadSnapshots(routeId : String) : ExternalRouteSnapshot[*]
-compareSnapshots(a : ExternalRouteSnapshot, b : ExternalRouteSnapshot) : Float
-computeConfidence(scores : Float[*]) : Float
-persistVerification(v : RouteVerification) : void
logs runs
ReliabilityService
+calculateRouteScore(routeId : String) : ReliabilityScore
+calculateBatch(routeIds : String[*]) : Integer
-loadHistory(routeId : String) : HistoricalPerformance[*]
-extractFactors(routeId : String) : Map
-applyWeights(factors : Map) : Float
-confidenceFromVerification(routeId : String) : String
-persistScore(score : ReliabilityScore) : void
SnapshotRepository
+saveAll(snaps : ExternalRouteSnapshot[*]) : Integer
+findByRoute(routeId : String) : ExternalRouteSnapshot[*]
VerificationRepository
+save(v : RouteVerification) : void
+latestForRoute(routeId : String) : RouteVerification
uses
ReliabilityRepository
+save(s : ReliabilityScore) : void
+latestForRoute(routeId : String) : ReliabilityScore
HistoryRepository
+insertMany(rows : HistoricalPerformance[*]) : Integer
+byRoute(routeId : String) : HistoricalPerformance[*]
RouteRepository
+getRoute(routeId : String) : Route
+listRoutes() : Route[*]
JobRunRepository
+start(jobName : String) : JobRun
+finish(runId : String, status : String, processed : Integer, err : String) : void
RouteNormaliser
+normaliseRoutes(raw : Json, sourceId : String) : ExternalRouteSnapshot[*]
+normaliseStations(raw : Json, sourceId : String) : Station[*]

Appendix B: Entity-Relation Diagram



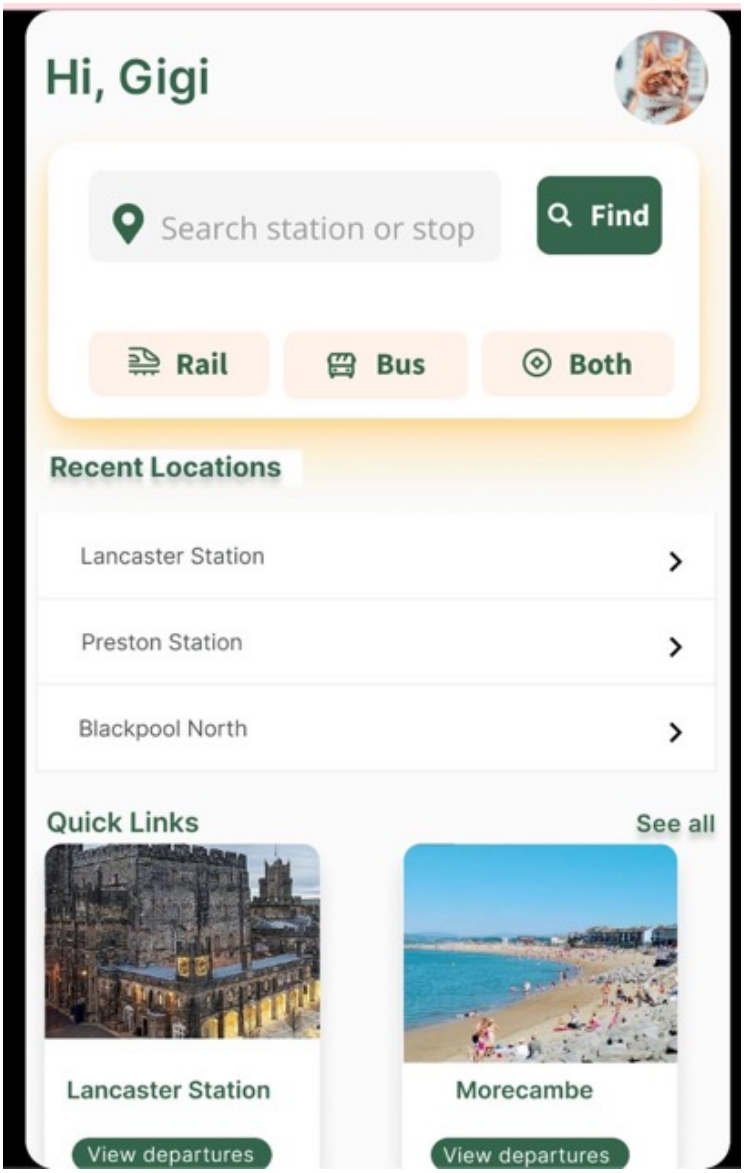
Appendix C: Database Table Relational Diagrams

Database Table Relational Diagram (Backend Role 1)

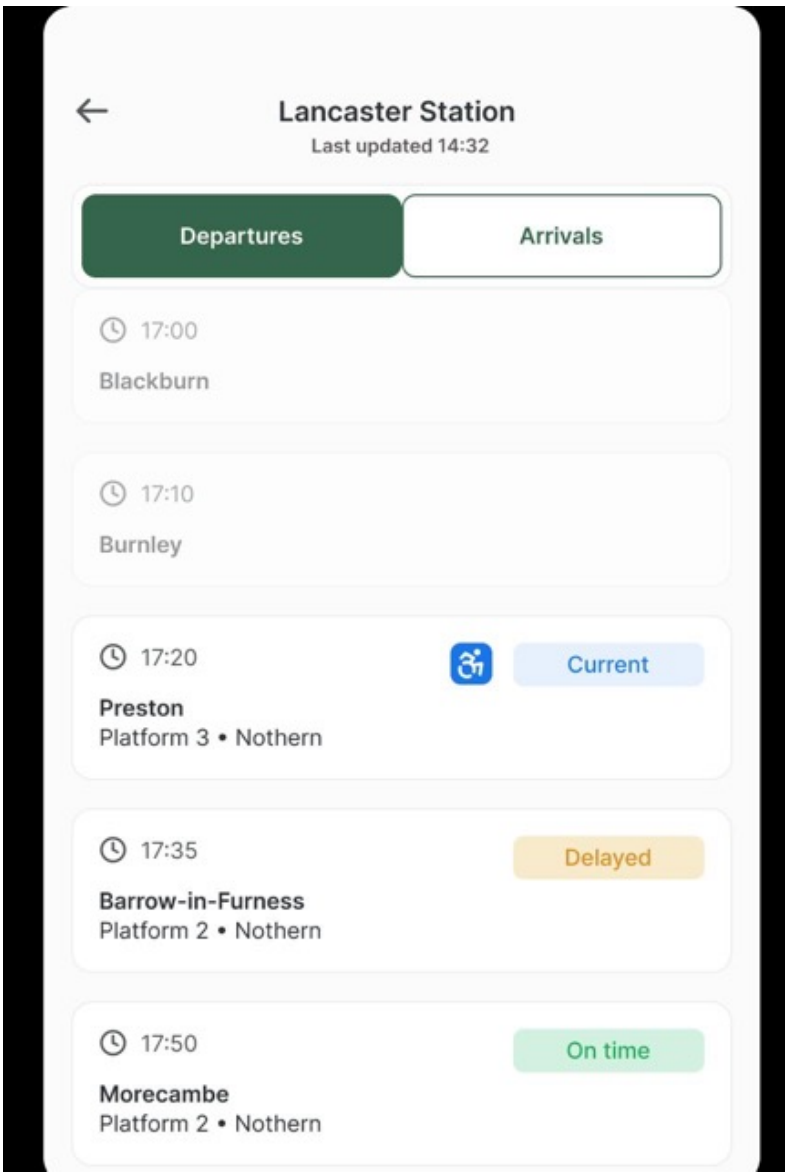


Appendix D: UI Mock-Up

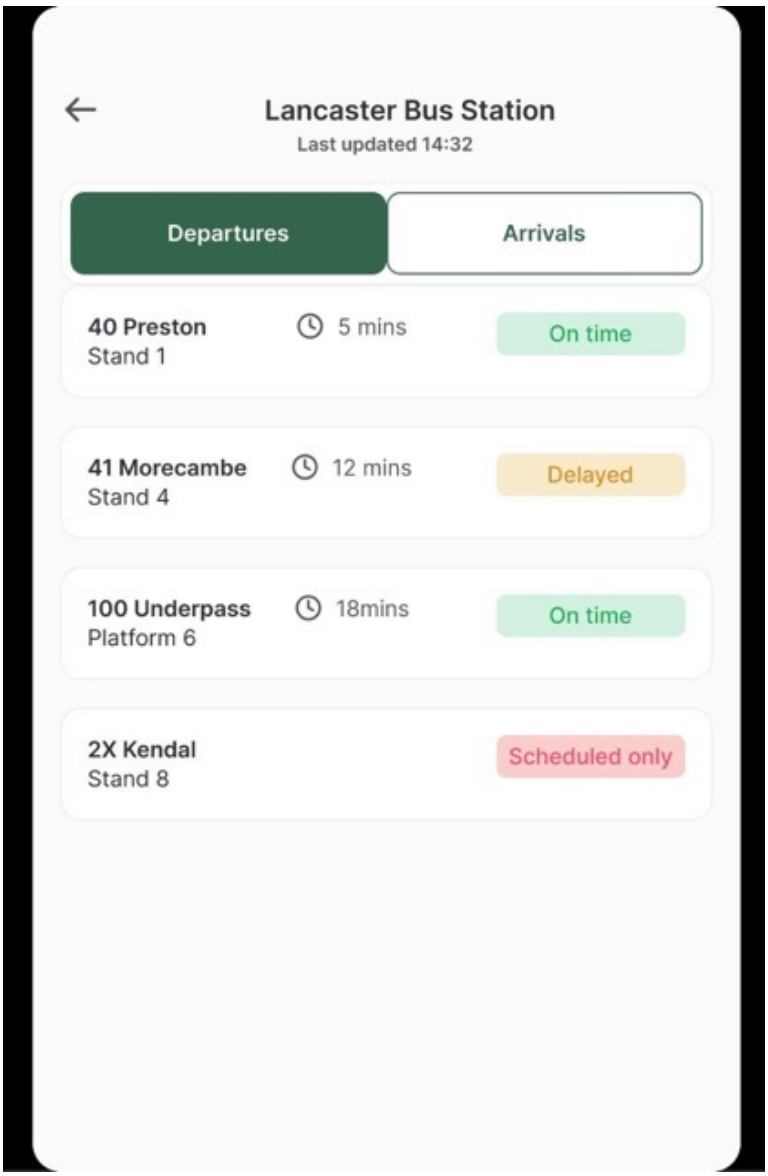
Main menu



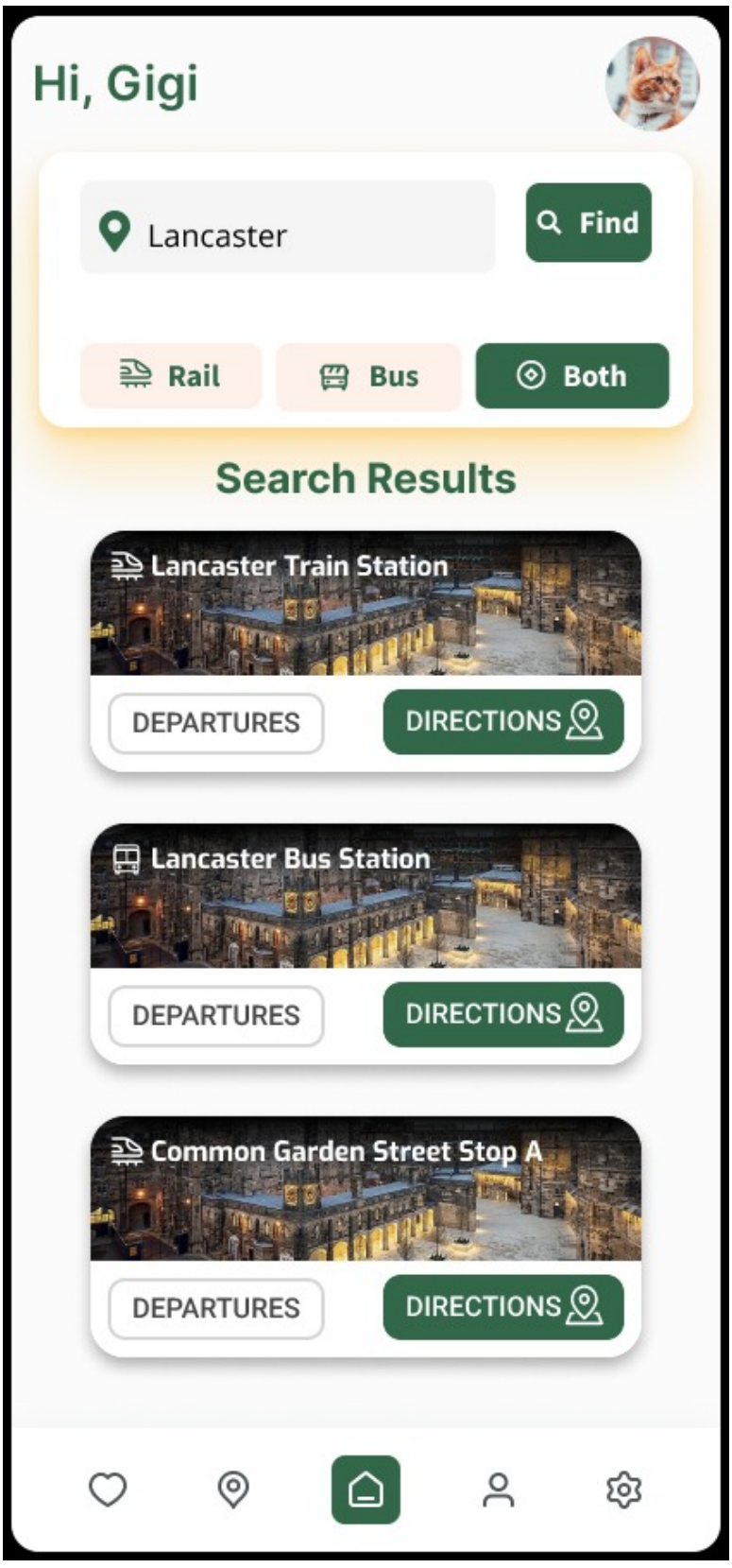
Train routing


Lancaster Station
Last updated 14:32
Departures
Arrivals
17:00
Blackburn
17:10
Burnley
17:20
Current
Preston
Platform 3 • Nothern
17:35
Delayed
Barrow-in-Furness
Platform 2 • Nothern
17:50
On time
Morecambe
Platform 2 • Nothern

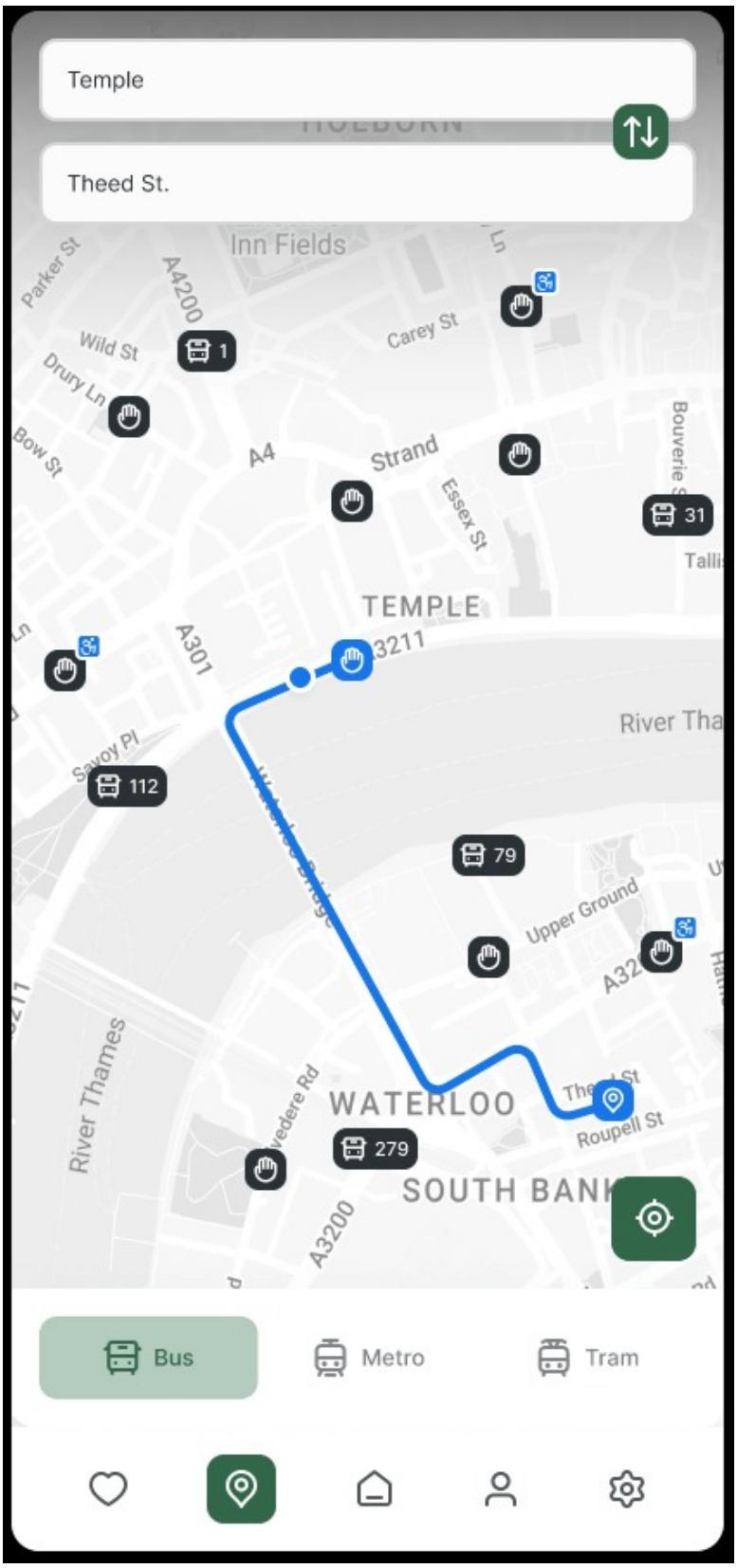
Bus routing



Train search


Hi, Gigi
Lancaster
Find
Rail
Bus
Both
Search Results
Lancaster Train Station
DEPARTURES
DIRECTIONS
Lancaster Bus Station
DEPARTURES
DIRECTIONS
Common Garden Street Stop A
DEPARTURES
DIRECTIONS

Map search



---

1. The total cost for each service assumes three years of support for the application. ↩ ↩2